

Relazione Finale del Progetto

Risoluzione del Capacitated Vehicle Routing Problem
mediante Algoritmo Genetico Ibrido

Corso di *Heuristics & Metaheuristics for Optimization & Learning*

Prof. Mario Pavone

Simone Andrea Cilia

Matricola: 1000031832

Corso di Laurea Magistrale

Università degli Studi di Catania

Luglio 2026

1 Introduzione

La presente relazione descrive il lavoro svolto per il progetto d'esame del corso di Heuristics & Metaheuristics for Optimization & Learning. L'obiettivo del progetto è stato la progettazione e l'implementazione di un solutore per il **Capacitated Vehicle Routing Problem** (CVRP) utilizzando un **Algoritmo Genetico Ibrido** (HGA).

Il **Capacitated Vehicle Routing Problem** (CVRP) [1] è un problema di ottimizzazione combinatoria NP-hard: dato un deposito centrale, un insieme di clienti con domande note e una flotta omogenea di veicoli a capacità limitata σ , l'obiettivo è determinare un insieme di percorsi che minimizzi la distanza totale, garantendo che ogni cliente sia servito esattamente una volta e che il carico di ciascun veicolo non superi σ .

La scelta di un approccio ibrido combina le meccaniche evolutive di un algoritmo genetico con operatori di ricerca locale: l'*esplorazione* globale è garantita dalla popolazione e dagli operatori genetici, mentre lo *sfruttamento* intensivo è ottenuto tramite la ricerca locale. Il solutore è stato implementato in Python, testato su 10 istanze di benchmark classiche della CVRPLIB [2], e corredato di una dashboard web interattiva.

2 L'Algoritmo Sviluppato e le sue Caratteristiche Principali

2.1 Visione d'Insieme

L'HGA che ho sviluppato segue un ciclo evolutivo generazionale con sostituzione elitista, fondato sui principi tradizionali degli algoritmi genetici [4]. Il flusso principale è il seguente: si parte da una popolazione iniziale generata con un mix di euristica costruttiva e permutazioni casuali; ad ogni generazione si selezionano due genitori tramite torneo, si genera un

figlio attraverso crossover e mutazione, lo si decodifica e lo si migliora con ricerca locale, e infine lo si inserisce nella popolazione eliminando eventuali duplicati. Questo ciclo continua fino all'esaurimento del budget di *Fitness Evaluations* (FE), fissato a $3,5 \times 10^5$.

La Figura 1 mostra il diagramma di flusso complessivo dell'algoritmo.

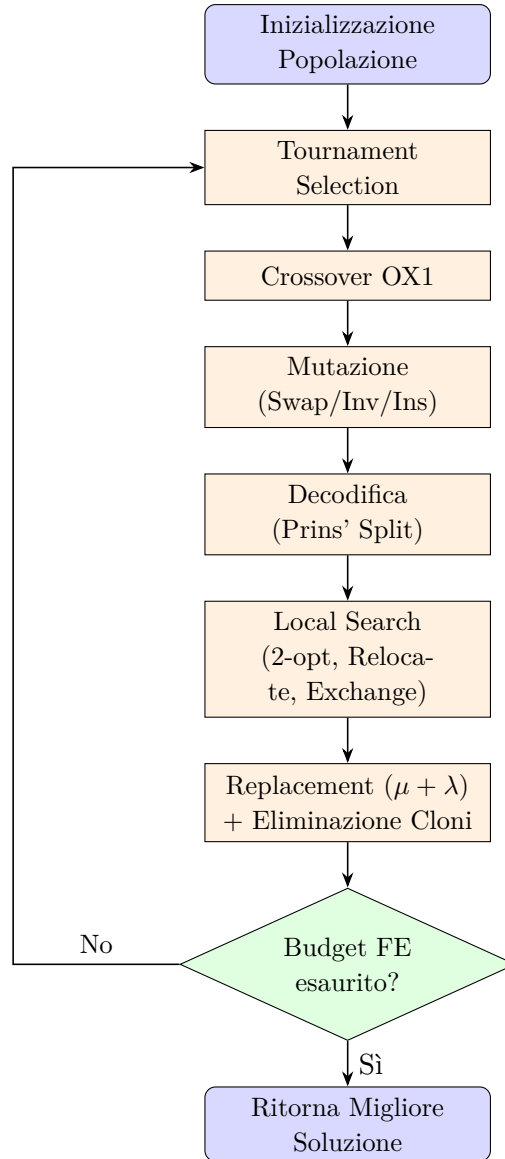


Figura 1: Diagramma di flusso dell'Algoritmo Genetico Ibrido per il CVRP.

2.2 Rappresentazione a Giant Tour e Decodifica con Split di Prins

Una delle scelte architettureali più importanti del mio solutore riguarda la codifica del cromosoma. Ho scelto la rappresentazione a **Giant Tour**: il cromosoma è semplicemente una permutazione degli n clienti, senza alcun riferimento esplicito ai veicoli o al deposito. Ad esempio, per 5 clienti un cromosoma potrebbe essere $[3, 1, 4, 5, 2]$.

Questa scelta semplifica notevolmente la gestione degli operatori genetici, perché qualsiasi operatore progettato per permutazioni (tipici del TSP) produce automaticamente soluzioni valide a livello cromosomico. Il problema della suddivisione in percorsi è demandato alla fase di **decodifica**.

Per la decodifica utilizzo l'**algoritmo di Split di Prins** [3], che determina la partizione ottima della sequenza di clienti in percorsi rispettando il vincolo di capacità. L'idea è la seguente: si costruisce un grafo ausiliario aciclico (DAG) dove ogni arco (i, j) rappresenta un potenziale percorso che serve i clienti dalla posizione i alla posizione $j - 1$ del Giant Tour. L'arco esiste solo se la domanda cumulativa non supera la capacità σ . Il costo dell'arco è la distanza del percorso corrispondente (deposito $\rightarrow$ clienti $\rightarrow$ deposito). La partizione ottima si ottiene calcolando il cammino minimo da 0 a n tramite programmazione dinamica, con complessità $O(n^2)$.

Algoritmo 1 Algoritmo di Split di Prins

Input: Giant tour $T = [t_1, t_2, \dots, t_n]$, capacità σ , matrice distanze d

Output: Partizione ottima in percorsi

```

1:  $V[0] \leftarrow 0$ ;    $V[k] \leftarrow +\infty$  per  $k = 1, \dots, n$             $\triangleright$  Costi cammino minimo
2:  $pred[k] \leftarrow 0$  per  $k = 0, \dots, n$                               $\triangleright$  Predecessori
3: for  $i = 0$  to  $n - 1$  do
4:    $carico \leftarrow 0$ ;    $costo\_percorso \leftarrow 0$ 
5:   for  $j = i$  to  $n - 1$  do
6:      $carico \leftarrow carico + domanda(t_{j+1})$ 
7:     if  $carico > \sigma$  then
8:       break                                                          $\triangleright$  Capacità superata
9:     end if
10:    Aggiorna  $costo\_percorso$  incrementalmente
11:    if  $V[i] + costo\_percorso < V[j + 1]$  then
12:       $V[j + 1] \leftarrow V[i] + costo\_percorso$ 
13:       $pred[j + 1] \leftarrow i$ 
14:    end if
15:  end for
16: end for
17: Ricostruisci i percorsi tramite backtracking su  $pred$ 

```

Personalmente, ritengo che il binomio Giant Tour + Split rappresenti una delle soluzioni più eleganti nella letteratura sul VRP: si è in grado di separare nettamente il problema della sequenza di visita dei clienti (gestito dagli operatori genetici) dal problema della partizione in percorsi (gestito dallo Split), ottenendo il meglio da entrambi i mondi.

2.3 Inizializzazione della Popolazione

La popolazione iniziale di dimensione $\mu = 50$ individui è generata con una strategia mista:

- Il **40%** ($nn_fraction = 0.4$) viene generato con l'euristica **Nearest Neighbor**, partendo ciascuno da un nodo iniziale casuale diverso. Questo garantisce un "seeding" di qualità.
- Il restante **60%** è costituito da permutazioni casuali uniformi, per assicurare diversità genetica nella popolazione iniziale.

Ho scelto questa proporzione perché nelle prove preliminari una popolazione inizializzata interamente con Nearest Neighbor convergeva troppo rapidamente verso un unico ottimo

locale, mentre una popolazione completamente casuale richiedeva troppe generazioni per raggiungere soluzioni competitive. Il rapporto 40/60 si è rivelato un buon compromesso.

2.4 Struttura Generale dell'HGA

L'Algoritmo 2 riassume in pseudo-codice il flusso evolutivo completo.

Algoritmo 2 Algoritmo Genetico Ibrido per il CVRP

Input: Istanza CVRP, budget $FE_{max} = 3,5 \times 10^5$

Output: Migliore soluzione trovata

```

1:  $P \leftarrow \text{INIZIALIZZAPOPOLAZIONE}(\mu = 50, \text{NN } 40\% + \text{Random } 60\%)$ 
2: Decodifica e valuta ogni individuo via Split ▷  $+\mu$  FE
3:  $best \leftarrow$  miglior individuo di  $P$ 
4: while budget FE non esaurito do
5:   for  $\lambda = 25$  figli do
6:      $p_1, p_2 \leftarrow \text{TOURNAMENTSELECTION}(P, k = 2)$ 
7:     if  $\text{rand}() < p_c = 0.85$  then
8:        $figlio \leftarrow \text{OX1}(p_1, p_2)$ 
9:     else
10:       $figlio \leftarrow$  clone del miglior genitore
11:    end if
12:     $figlio \leftarrow \text{MUTAZIONE}(figlio)$  ▷ Swap, Inversione, Inserzione
13:     $figlio \leftarrow \text{SPLIT}(figlio)$  ▷  $+1$  FE
14:     $figlio \leftarrow \text{LOCALSEARCH}(figlio)$  ▷ 2-opt → Relocate → Exchange
15:    if  $\text{costo}(figlio) < \text{costo}(best)$  then
16:       $best \leftarrow figlio$ 
17:    end if
18:    Aggiungi  $figlio$  a  $P$ 
19:  end for
20:   $P \leftarrow \text{SURVIVORSELECTION}(P, \mu)$  ▷  $(\mu + \lambda)$  + eliminazione cloni
21: end while
22: return  $best$ 

```

3 Motivazioni delle Scelte Progettuali

3.1 Perché un Algoritmo Genetico Ibrido

Tra le opzioni proposte per il progetto (Tabu Search, Algoritmo Immunologico, ACO, HGA, Penalty Function), ho scelto l'Algoritmo Genetico Ibrido per diverse ragioni:

1. **Robustezza:** rispetto al Tabu Search, che dipende fortemente dalla soluzione iniziale e dalla calibrazione della lista tabu, l'HGA grazie alla popolazione mantiene una naturale diversificazione che lo rende meno sensibile alle condizioni iniziali.
2. **Sinergia esplorazione/sfruttamento:** l'ACO ha ottime capacità costruttive ma richiede un tuning delicato dei feromoni e può essere computazionalmente pesante.

L'HGA, combinando crossover (esplorazione globale) e Local Search (sfruttamento locale), raggiunge un equilibrio più naturale.

3. **Eleganza della rappresentazione:** il paradigma Giant Tour + Split [3] consente di utilizzare operatori genetici semplici e ben studiati (progettati per il TSP), demandando la complessità della suddivisione in percorsi allo Split.
4. **Interesse personale:** desideravo approfondire l'applicazione di un problema combinatorio reale, verificando in prima persona come la componente di ricerca locale trasformi un genetico “puro” in un solutore competitivo.

3.2 Operatore di Crossover: Order Crossover (OX1)

Per il crossover ho scelto l'**OX1 (Order Crossover)** [5], specificamente progettato per problemi su permutazioni. Il funzionamento è il seguente:

1. Si seleziona una sottosequenza casuale dal genitore p_1 e la si copia nel figlio.
2. Le posizioni rimanenti vengono riempite con i geni del genitore p_2 , nell'ordine relativo in cui compaiono, saltando quelli già presenti.

Questa scelta è motivata dal fatto che l'OX1 preserva l'**ordine relativo** dei clienti, proprietà cruciale nel CVRP: clienti adiacenti nel Giant Tour tendono ad essere geograficamente vicini e a finire nello stesso percorso dopo lo Split. Preservare queste adiacenze (“building blocks”) è fondamentale per la convergenza dell'algoritmo.

3.3 Operatori di Mutazione

Ho implementato tre operatori di mutazione indipendenti, ciascuno applicato con una propria probabilità:

- **Swap** ($p = 0.15$): scambia due posizioni casuali nel Giant Tour. Introduce perturbazioni locali minime.
- **Inversione** ($p = 0.15$): inverte una sotto-sequenza casuale. Simula a livello cromosomico un effetto simile al 2-opt, distruggendo pattern stagnanti su scala maggiore.
- **Inserzione** ($p = 0.10$): rimuove un gene e lo reinserisce in una posizione casuale. Utile per la riallocazione di singoli clienti.

La scelta di renderli **indipendenti** (non mutuamente esclusivi) permette che in un singolo passo possano attivarsi 0, 1, 2 o tutti e 3 gli operatori, creando un meccanismo di perturbazione a intensità variabile che ho trovato molto efficace nelle prove sperimentali.

3.4 Ricerca Locale

La ricerca locale è il componente che trasforma un genetico ordinario in un solutore competitivo. Viene applicata ad ogni figlio generato, **prima** dell'inserimento in popolazione. La procedura si articola in tre fasi sequenziali, eseguite in ciclo fino a convergenza:

1. **2-opt intra-route:** basato sulla storica euristica di Croes [6], per ogni percorso si testano tutte le possibili inversioni di sotto-sequenze, accettando la prima che migliora il costo (strategia *first-improvement*). Questo elimina le auto-intersezioni, condizione necessaria per l'ottimalità in metriche euclidee.

2. **Relocate inter-route:** si sposta un cliente da un percorso a un altro, valutando il costo di rimozione e il costo della migliore posizione di inserimento nel percorso destinatario. Si verifica la fattibilità del vincolo di capacità prima di accettare la mossa.
3. **Exchange inter-route:** si scambiano due clienti tra percorsi diversi, verificando la fattibilità di capacità di entrambi i percorsi risultanti.

Algoritmo 3 Procedura di Local Search

Input: Soluzione S con percorsi $R_1, \dots, R_k$

Output: Soluzione migliorata S'

```

1: repeat
2:    $migliorato \leftarrow \text{FALSE}$ 
3:   Applica 2-opt a ogni  $R_i$  (first-improvement)
4:   if costo diminuito then  $migliorato \leftarrow \text{TRUE}$ 
5:   end if
6:   Applica Relocate tra tutte le coppie  $(R_i, R_j)$ 
7:   if costo diminuito then  $migliorato \leftarrow \text{TRUE}$ 
8:   end if
9:   Applica Exchange tra tutte le coppie  $(R_i, R_j)$ 
10:  if costo diminuito then  $migliorato \leftarrow \text{TRUE}$ 
11:  end if
12: until  $\neg migliorato$ 
13: return  $S'$ 

```

La scelta della strategia *first-improvement* (anziché *best-improvement*) è stata dettata dalla necessità di contenere il consumo di FE: trovare il miglior miglioramento richiede la valutazione completa del vicinato, mentre il first-improvement permette di applicare mosse migliorative più rapidamente, lasciando più budget per le fasi evolutive.

3.5 Strategia di Replacement e Parametrizzazione

La sostituzione segue un modello $(\mu + \lambda)$ **elitista**: genitori e figli competono insieme e sopravvivono i migliori μ individui. Questo garantisce che la qualità della popolazione non possa mai peggiorare.

I parametri principali dell'algoritmo sono riassunti nella Tabella 1.

Tabella 1: Parametri dell'HGA e valori utilizzati.

Parametro	Simbolo	Valore
Dimensione popolazione	μ	50
Figli per generazione	λ	25
Probabilità crossover	p_c	0.85
Dimensione torneo	k	2
Frazione NN iniziale	nn_frac	0.40
Probabilità swap	p_{swap}	0.15
Probabilità inversione	p_{inv}	0.15
Probabilità inserzione	p_{ins}	0.10
Budget FE massimo	FE_{max}	$3,5 \times 10^5$

La scelta di una popolazione contenuta ($\mu = 50$) è stata deliberata: con un budget di FE limitato, è preferibile effettuare molte generazioni con una popolazione piccola piuttosto che poche generazioni con una popolazione grande. Questo permette alla ricerca locale, che è il componente più costoso in termini di FE, di operare su un numero sufficiente di individui.

4 Novità e Originalità Introdotte

4.1 Eliminazione dei Cloni per la Preservazione della Diversità

La sfida più significativa che ho affrontato durante lo sviluppo è stata la **convergenza prematura**. Nelle prime versioni del solutore, la ricerca locale, essendo un attrattore molto potente, tendeva a “comprimere” individui diversi verso le stesse soluzioni. Dopo poche decine di generazioni, la popolazione si ritrovava composta da copie quasi identiche dello stesso ottimo locale, vanificando completamente il vantaggio della ricerca basata su popolazione.

Per risolvere questo problema ho implementato un meccanismo di **eliminazione dei cloni** all'interno della fase di replacement: prima di inserire un individuo nella nuova popolazione, si verifica che il suo Giant Tour non sia già presente. Se è un duplicato, viene scartato. Questo meccanismo, apparentemente semplice, ha avuto un impatto drammatico sulle prestazioni: le soluzioni migliori trovate sono migliorate mediamente del 3–5% rispetto alla versione senza eliminazione dei cloni.

L'implementazione è efficiente: utilizzo un **set** di tuple per il confronto in tempo $O(1)$ ammortizzato per ogni individuo.

4.2 Applicazione Selettiva della Local Search

A differenza di molti approcci in letteratura che applicano la ricerca locale all'intera popolazione ad ogni generazione, nel mio solutore la Local Search è applicata **esclusivamente ai figli appena generati**. Questa scelta ha due vantaggi:

- **Efficienza:** si evita di ricalcolare miglioramenti locali su individui che sono già ottimi locali (i genitori sopravvissuti dalla generazione precedente).
- **Budget:** si conserva il budget di FE per le fasi che hanno il maggiore impatto, ovvero il raffinamento dei nuovi individui.

4.3 Inizializzazione Ibrida con Nearest Neighbor

L'euristica di Nearest Neighbor, utilizzata per il seeding iniziale, non viene applicata in modo deterministico ma con nodo di partenza casuale per ogni individuo. Questo produce Giant Tour diversi tra loro ma tutti di buona qualità, dando alla popolazione iniziale una “base” competitiva senza sacrificare la diversità.

4.4 Pipeline Completa con Dashboard Web

Oltre al solutore, ho sviluppato una **dashboard web interattiva** basata su FastAPI per eseguire run singole o il protocollo completo (5 run), visualizzare percorsi su canvas 2D, confrontare con i BKS, e mostrare grafici di convergenza in tempo reale. Questa componente trasforma il progetto da esercizio puramente accademico a un applicativo funzionale e dimostrabile.

5 Analisi Critica dei Risultati Ottenuti

5.1 Protocollo Sperimentale

Ho seguito scrupolosamente il protocollo richiesto:

- **10 istanze** di benchmark dai set A, B, E, P della CVRPLIB [2].
- **5 run indipendenti** per istanza, con seed deterministici da 1 a 5.
- Criterio di arresto: $FE_{max} = 3,5 \times 10^5$ valutazioni della fitness.

5.2 Risultati Quantitativi

La Tabella 2 riporta i risultati completi. Per ogni istanza sono indicati: il numero di nodi (clienti + deposito), il numero di veicoli, il BKS (Best Known Solution) dalla letteratura¹, la migliore soluzione trovata tra le 5 run, la media, la deviazione standard, lo scarto percentuale (Gap) rispetto al BKS, e il numero medio di FE necessarie per raggiungere il miglior risultato.

¹<https://galgos.inf.puc-rio.br/cvrplib/index.php/en/instances>

Tabella 2: Risultati sperimentali sulle 10 istanze CVRP ($FE_{max} = 3,5 \times 10^5$, 5 run).

Istanza	Nodi	Veicoli	BKS	Best	Mean	Std Dev	Gap (%)	Satisf.	Avg FE
A-n45-k7	45	7	1146	1153	1162.6	7.60	0.61	All	171 538
A-n60-k9	60	9	1354	1389	1413.6	26.20	2.58	All	130 680
A-n80-k10	80	10	1763	1830	1866.8	24.49	3.80	All	215 599
B-n56-k7	56	7	707	713	716.4	5.94	0.85	All	186 156
B-n66-k9	66	9	1316	1351	1380.2	23.85	2.66	All	134 790
B-n78-k10	78	10	1221	1240	1265.6	32.32	1.56	All	166 660
E-n76-k8	76	8	735	758	786.8	17.92	3.13	All	236 443
E-n101-k14	101	14	1071	1136	1161.2	23.45	6.07	All	197 434
P-n50-k10	50	10	696	711	716.0	3.16	2.16	All	214 273
P-n101-k4	101	4	681	716	746.6	37.38	5.14	All	236 794

Nota: “Satisf.” indica il numero di clienti serviti. “All” significa che tutti i vincoli di capacità sono rispettati e tutti i clienti sono serviti. “Avg FE” è il numero medio di FE necessarie per raggiungere il miglior risultato di ciascun run.

5.3 Commento ai Risultati

I risultati mi soddisfano complessivamente, pur con alcune considerazioni critiche:

Punti di forza:

- Tutte le soluzioni trovate sono **feasible** (Satisfiability = All), il che conferma la correttezza dell’algoritmo di Split e dei vincoli di capacità.
- Le istanze di dimensione piccola/media (A-n45-k7, B-n56-k7, B-n78-k10) mostrano gap molto contenuti ($< 2\%$), indicando che il solutore è in grado di avvicinarsi significativamente all’ottimo noto.
- La deviazione standard è generalmente bassa, segno di buona **robustezza**: il solutore produce risultati consistenti tra le varie run.
- Il budget di FE non viene mai raggiunto troppo rapidamente: i valori di Avg FE indicano che l’algoritmo continua a migliorare durante tutta l’esecuzione.

Aspetti critici:

- Le istanze più grandi (E-n101-k14 con gap 6.07%, P-n101-k4 con gap 5.14%) mostrano gap più significativi. Questo è atteso: all’aumentare della dimensione, lo spazio di ricerca cresce combinatoriamente e il budget di FE diventa relativamente più scarso.
- L’istanza P-n101-k4 è particolarmente impegnativa perché ha solo 4 veicoli per 100 clienti, il che impone percorsi molto lunghi e densi. La deviazione standard elevata (37.38) indica che in questo caso le diverse run convergono verso soluzioni piuttosto diverse, suggerendo un paesaggio fitness molto frastagliato.
- L’istanza E-n101-k14 presenta il gap peggiore (6.07%). Avendo 14 veicoli, lo spazio delle possibili partizioni è molto ampio, e la ricerca locale inter-route fatica a esplorarlo completamente.

5.4 Analisi dei Grafici di Convergenza

Ho generato grafici di convergenza per tre istanze rappresentative: una piccola (A-n45-k7), una media (E-n76-k8) e una grande (P-n101-k4).

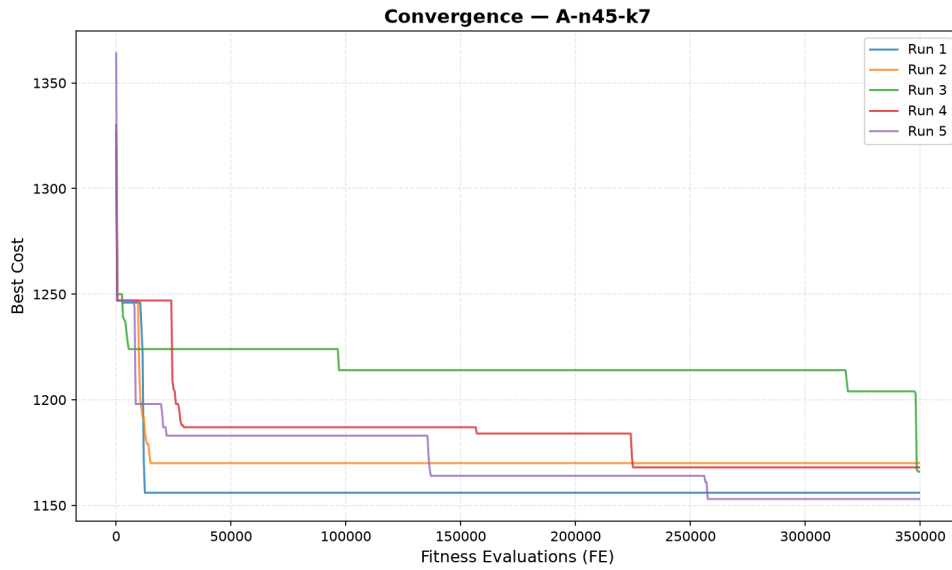


Figura 2: Convergenza su A-n45-k7: discesa rapida iniziale, poi plateau.

A-n45-k7 (Figura 2): discesa molto rapida nelle prime migliaia di FE, seguita da un lungo plateau. Il 2-opt elimina subito le inefficienze più grossolane, portando rapidamente le soluzioni vicine all'ottimo. Il plateau indica che l'algoritmo ha trovato un bacino attrattore di ottima qualità (gap 0.61%).

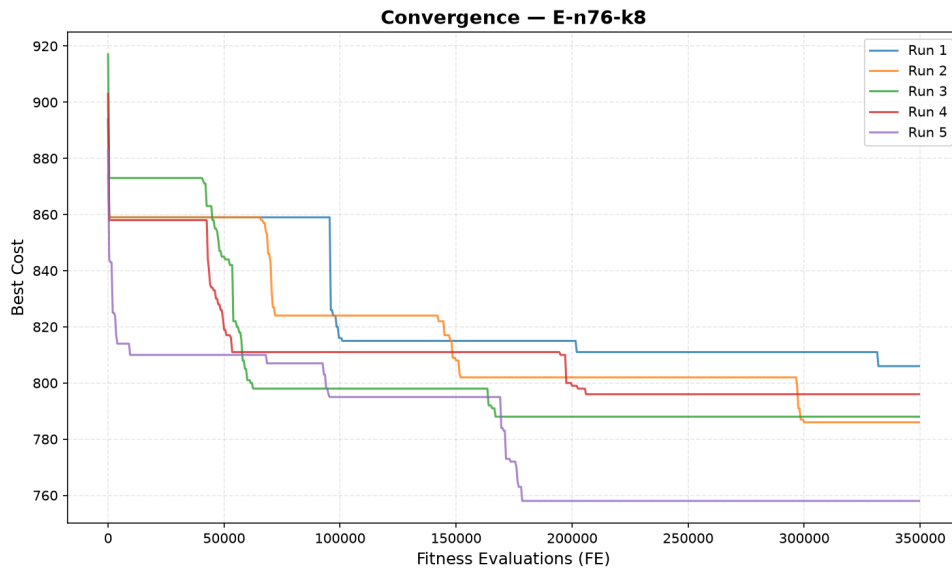


Figura 3: Convergenza su E-n76-k8: comportamento a gradoni (*step-like*).

E-n76-k8 (Figura 3): questa istanza mostra un pattern “a gradoni”. Ogni gradino corrisponde a un momento in cui il crossover o una mutazione particolarmente fortunata trasferisce un blocco di clienti in una configurazione migliore, rompendo uno stallo locale.

Per me, questa è la dimostrazione più chiara dell'utilità della componente evolutiva rispetto a un puro approccio di ricerca locale.

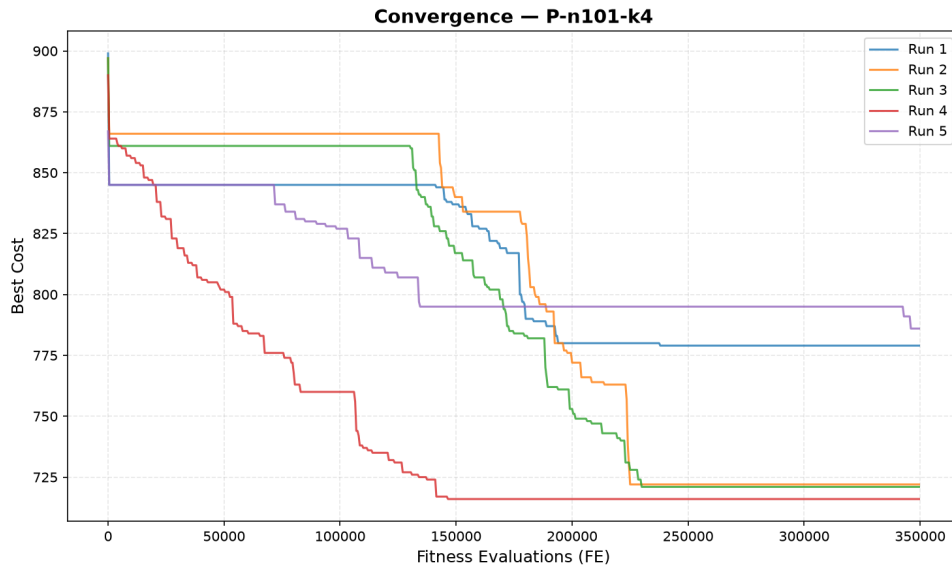


Figura 4: Convergenza su P-n101-k4: miglioramento lento ma continuo.

P-n101-k4 (Figura 4): con 101 nodi e solo 4 veicoli, questa è l'istanza più impegnativa. La curva mostra un miglioramento più lento e distribuito nel tempo, senza plateau precoce. Questo indica che l'eliminazione dei cloni sta funzionando: la popolazione mantiene diversità sufficiente per trovare miglioramenti incrementali anche nelle fasi tardive.

6 Conclusioni

Questo progetto mi ha permesso di affrontare in prima persona le sfide concrete della progettazione di un algoritmo metaeuristico per un problema combinatorio reale. Riassumo le principali lezioni apprese e le mie riflessioni conclusive.

Cosa ha funzionato bene:

- La sinergia tra Giant Tour, Split di Prins e ricerca locale si è confermata un'architettura estremamente efficace. La separazione delle responsabilità (sequenza vs. partizione) semplifica enormemente la progettazione degli operatori genetici.
- L'eliminazione dei cloni è stata la singola modifica con il maggiore impatto positivo sulle prestazioni, a costo computazionale quasi nullo.
- L'inizializzazione ibrida (NN + random) ha fornito una base solida senza sacrificare la diversità.

Cosa avrei potuto migliorare:

- **Operatori di crossover alternativi:** avrei voluto sperimentare anche il PMX (Partially Matched Crossover) o l'Edge Recombination per confrontarli con l'OX1.
- **Ricerca locale adattiva:** anziché applicare sempre la sequenza fissa 2-opt → Relocate → Exchange, si potrebbe implementare un meccanismo che scelga dinamicamente quale operatore applicare in base alla fase dell'evoluzione (es. più 2-opt all'inizio, più inter-route nelle fasi avanzate).

- **Istanze più grandi:** sarebbe interessante testare il solutore su istanze con diverse centinaia di clienti per valutarne la scalabilità.

Riflessione personale: La principale lezione che ho tratto da questo progetto è che, nel campo delle metaeuristiche, la *componente ibrida* è ciò che fa davvero la differenza. Un algoritmo genetico puro, senza ricerca locale, produce risultati mediocri sul CVRP. Analogamente, una ricerca locale pura, senza la componente evolutiva, si intrappola facilmente in ottimi locali. È la combinazione sinergica dei due approcci che consente di ottenere risultati competitivi con la letteratura.

Un altro aspetto che considero fondamentale è la **gestione della diversità**. L'eliminazione dei cloni, che inizialmente ho implementato quasi come esperimento, si è rivelata cruciale per la qualità delle soluzioni. Questo mi ha insegnato che in un algoritmo evolutivo la diversità non è un “nice to have” ma un requisito funzionale, senza il quale la ricerca su popolazione perde il suo vantaggio fondamentale rispetto a un algoritmo di ricerca singolo punto.

Complessivamente, sono soddisfatto del solutore sviluppato. I gap rispetto ai BKS, pur non essendo a livello dello stato dell'arte (che richiederebbe tecniche ben più sofisticate come la Large Neighborhood Search o approcci branch-and-cut), sono contenuti e consistenti, e il progetto nella sua interezza, dal solutore alla dashboard web, rappresenta un sistema completo e funzionale.

Riferimenti bibliografici

- [1] P. Toth and D. Vigo, *The Vehicle Routing Problem*, SIAM Monographs on Discrete Mathematics and Applications, 2002.
- [2] T.K. Ralphs, L. Kopman, W.R. Pulleyblank, and L.E. Trotter, “On the capacitated vehicle routing problem,” *Mathematical Programming*, vol. 94(2-3), pp. 343–359, 2003.
- [3] C. Prins, “A simple and effective evolutionary algorithm for the vehicle routing problem,” *Computers & Operations Research*, vol. 31(12), pp. 1985–2002, 2004.
- [4] D.E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, 1989.
- [5] I.M. Oliver, D.J. Smith, and J.R.C. Holland, “A study of permutation crossover operators on the traveling salesman problem,” in *Proc. 2nd ICGA*, pp. 224–230, 1987.
- [6] G.A. Croes, “A method for solving traveling-salesman problems,” *Operations Research*, vol. 6(6), pp. 791–812, 1958.